COLOR GUESSING GAME DEVELOPMENT USING PYTHON

AIM

To design and develop an interactive 2D Color Guessing Game using Python Tkinter that starts with a START button, displays a colour on the screen, and allows the player to identify its correct name from three options by clicking buttons, earn points, manage limited lives within a time limit, and view the final score, star rating and high score. The game includes a graphical interface, score tracking, lives and timer handling, game-over detection, and Start/Restart and Exit options. The colour box and option boxes are hidden on the start screen, and the option boxes are hidden on the game-over screen, so only the relevant information is shown.

PROCEDURE

1. Start the program and import the required Python libraries: tkinter and random.
2. Create a dictionary of 20 colour names with their hexadecimal colour codes (Red, Blue, Green, Yellow, Purple, Orange, Pink, Brown, Cyan, Magenta, Lime, Navy, Gold, Coral, Violet, Teal, Indigo, Salmon, Olive and Chocolate).
3. Initialize the game variables: score, high score, lives (3) and time limit (20 seconds). Set game_over to True so that the game waits until the START button is pressed.
4. Create the main game window using Tkinter, set the title as "Color Guessing Game", and fix the window size at 380x600.
5. Design the graphical interface using Label widgets for the game title, timer, lives, colour display box, message area, score and high score. The colour display box is placed in a fixed-size frame and stays hidden until the game starts.
6. Create three option buttons using the Button widget inside a fixed-size frame to display the colour names. They stay hidden until the game starts and are hidden again when the game ends.
7. Create the START and RESTART buttons, which call start_game() to reset the score, lives and timer, show the colour box and option buttons and begin the first round and the countdown. START is disabled while a game is running, and RESTART stays disabled until the first game starts.
8. Create the EXIT button, which closes the game window using root.destroy.
9. Implement the new_round() function to select a random target colour, display it in the colour box, and generate three shuffled options that always include the correct answer.
10. Implement the check() function to compare the selected button with the target colour. For a correct answer, increase the score and start the next round after 700 ms using the after() method. For a wrong answer, reduce one life.
11. Implement the countdown() function using Tkinter's after() method to update the timer every second.
12. Implement the end_game() function to stop the game when the player scores 5 points, loses all lives, or the time reaches zero. Display the star rating, result message, final score and high score, and hide the three option buttons (the "-" boxes) so the game-over screen stays clean, just like the start screen.
13. Execute the game and verify the START and EXIT buttons, colour display, option buttons, scoring, lives, timer, game-over detection (with the option boxes hidden) and restart functionality.

PROGRAM CODE (Part 1 of 4)

```
import tkinter as tk, random

colors = {
    "Red": "#FF0000", "Blue": "#0000FF", "Green": "#008000",
    "Yellow": "#FFFF00", "Purple": "#800080", "Orange": "#FFA500",
    "Pink": "#FFC0CB", "Brown": "#A52A2A", "Cyan": "#00FFFF",
    "Magenta": "#FF00FF", "Lime": "#00FF00", "Navy": "#000080",
    "Gold": "#FFD700", "Coral": "#FF7F50", "Violet": "#EE82EE",
    "Teal": "#008080", "Indigo": "#4B0082", "Salmon": "#FA8072",
    "Olive": "#808000", "Chocolate": "#D2691E",
}

score, high_score, lives = 0, 0, 3
time_left, game_over, target, jobs = 20, True, "", []


def new_round():
    global target
    if game_over:
        return
    target = random.choice(list(colors))
    color_label.config(text=colors[target], bg=colors[target], fg="black")
    options = random.sample(list(colors), 3)
    if target not in options:
        options[random.randrange(3)] = target
    random.shuffle(options)
    for b, o in zip(buttons, options):
        b.config(text=o, state="normal", command=lambda c=o: check(c))
    message.config(text="Choose the correct color!")
```

PROGRAM CODE (Part 2 of 4)

```
def check(choice):
    global score, lives
    if game_over:
        return
    if choice == target:
        score += 1
        score_label.config(text=f"Score: {score}/5")
        for b in buttons:
            b.config(state="disabled")
        if score == 5:
            end_game()
        else:
            message.config(text="Correct! Next color...")
            jobs.append(root.after(700, new_round))
    else:
        lives -= 1
        lives_label.config(text=f"Lives: {lives}")
        message.config(text="Wrong! Try again.")
        if lives == 0:
            end_game()


def countdown():
    global time_left
    if game_over:
        return
    timer_label.config(text=f"Time: {time_left} sec")
    if time_left == 0:
        end_game()
        return
    time_left -= 1
    jobs.append(root.after(1000, countdown))
```

PROGRAM CODE (Part 3 of 4)

```
def end_game():
    global game_over, high_score
    if game_over:
        return
    game_over = True
    while jobs:
        root.after_cancel(jobs.pop())
    high_score = max(high_score, score)
    stars = ("★★★" if score == 5 else
             "★★" if score >= 3 else "★" * (score > 0))
    result = ["Keep trying! Your next win is waiting.", "Good try!", "Good!",
              "Excellent! All 5 colors!"][len(stars)]
    color_label.config(text=stars or "GAME OVER", bg="#F3F4F6", fg="#F59E0B")
    message.config(text=result)
    score_label.config(text=f"Final Score: {score}/5")
    best_label.config(text=f"High Score: {high_score}")
    for b in buttons:
        b.pack_forget()      # hide the "-" option boxes on the game-over screen
    start_btn.config(state="normal")


def start_game():  # used by START and RESTART buttons
    global score, lives, time_left, game_over
    while jobs:
        root.after_cancel(jobs.pop())
    score, lives, time_left, game_over = 0, 3, 20, False
    score_label.config(text="Score: 0/5")
    lives_label.config(text="Lives: 3")
    start_btn.config(state="disabled")
    restart_btn.config(state="normal")
    color_label.pack(pady=3)     # show the colour box
    for b in buttons:            # show the option buttons (in the same order)
        b.pack_forget()
        b.pack(pady=4)
    new_round()
    countdown()


def label(text, **kw):
    lbl = tk.Label(root, text=text, bg=kw.pop("bg", "#F3F4F6"), **kw)
    lbl.pack(pady=3)
    return lbl
```

PROGRAM CODE (Part 4 of 4)

```
root = tk.Tk()
root.title("Color Guessing Game")
root.geometry("380x600")
root.resizable(False, False)
root.configure(bg="#F3F4F6")

label("COLOR GUESSING GAME", font=("Arial", 18, "bold"))
timer_label = label("Time: 20 sec")
lives_label = label("Lives: 3")

# fixed-size frame keeps the layout steady while the colour box is hidden
color_frame = tk.Frame(root, bg="#F3F4F6", width=380, height=70)
color_frame.pack_propagate(False)
color_frame.pack()
color_label = tk.Label(color_frame, width=15, height=3,
                       font=("Arial", 12), bg="#E5E7EB")
# the colour box stays hidden on the start screen; start_game() shows it

message = label("Press START to begin", font=("Arial", 11))

# fixed-size frame keeps the layout steady while the option buttons are hidden
btn_frame = tk.Frame(root, bg="#F3F4F6", width=380, height=145)
btn_frame.pack_propagate(False)
btn_frame.pack()
buttons = [tk.Button(btn_frame, text="-", width=15, font=("Arial", 12),
                     state="disabled") for _ in range(3)]
# the option buttons stay hidden on the start screen; start_game() shows them

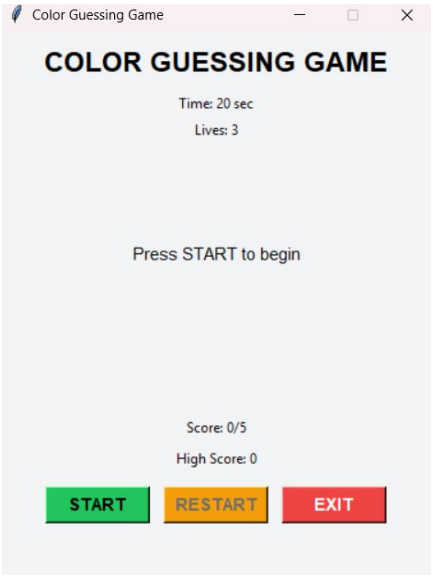
score_label = label("Score: 0/5")
best_label = label("High Score: 0")

frame = tk.Frame(root, bg="#F3F4F6")
frame.pack(pady=12)
buttons_info = (("START", "#22C55E", start_game),
                ("RESTART", "#F59E0B", start_game),
                ("EXIT", "#EF4444", root.destroy))
start_btn, restart_btn, exit_btn = [
    tk.Button(frame, text=t, bg=c, width=9, command=f,
              font=("Arial", 11, "bold"))
    for t, c, f in buttons_info]
restart_btn.config(state="disabled")
exit_btn.config(fg="white")
for b in (start_btn, restart_btn, exit_btn):
    b.pack(side="left", padx=6)

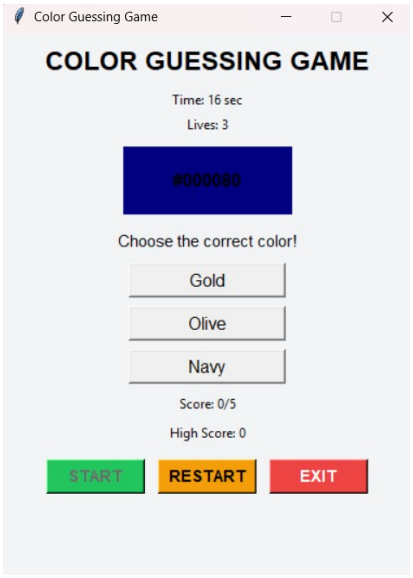
root.mainloop()
```

OUTPUT

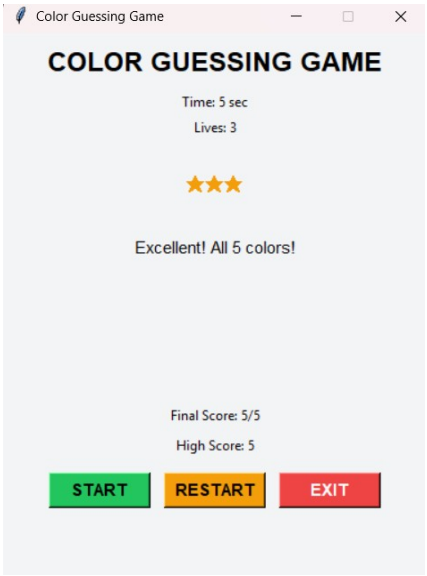
Screen 1: Start screen



Screen 2: Game in progress



Screen 3: Game over



RESULT

Thus, an interactive 2D Color Guessing Game was developed using Python Tkinter with graphical game design, Start/Restart and Exit buttons, random colour generation, button-based answer selection, automatic score calculation, lives and countdown timer handling, game-over detection with star ratings and a clean result screen, and high score tracking. The program demonstrates event-driven programming, GUI design, timed events using the after() method, conditional statements, and basic game development concepts.